

HDOC Day-35

Menu-Driven C Program Using Functions and Square Matrix

Problem Statement

Write a menu-driven C program to read a variable-length square integer array and perform the following operations using separate functions:

1. Determine whether all the elements on its main diagonal are distinct.
2. Find the sum of its main diagonal elements.
3. Display its elements using a diagonal traversal. Begin with the element in the top-left corner and traverse successive diagonals alternately in the upward-right and downward-left directions.
4. Exit the program.

Algorithm

1. Start.
2. Read the order n of the square matrix.
3. Read all n x n integer elements of the matrix.
4. Display the four menu options.
5. Read the user choice.
6. Use a switch statement: Choice 1 calls a function to check whether the main diagonal elements are distinct; Choice 2 calls a function to calculate the sum of the main diagonal elements; Choice 3 calls a function to display the matrix in diagonal traversal order; Choice 4 exits; default displays an invalid-choice message.
7. Repeat the menu until Choice 4 is selected.
8. Stop.

Test-Case Table

Test	Matrix / n	Choice	Expected Result
1	n = 3 1 2 3 4 5 6 7 8 9	1	Main diagonal elements 1, 5 and 9 are reported as distinct.
2	Same matrix	2	Sum of main diagonal elements is displayed as 15.
3	Same matrix	3	Diagonal traversal is displayed as: 1 2 4 7 5 3 6 8 9.
4	Same matrix	4	Program exits.
5	Same matrix	7	Invalid menu choice message is displayed and the menu appears again.

C Program

```
#include <stdio.h>
```

```
int distinctDiagonal(int n, int a[n][n])
{
    int i, j;
    for (i = 0; i < n; i++)
    {
        for (j = i + 1; j < n; j++)
        {
            if (a[i][i] == a[j][j])
                return 0;
        }
    }
    return 1;
}
```

```
int sumDiagonal(int n, int a[n][n])
{
    int i, sum = 0;
    for (i = 0; i < n; i++)
        sum += a[i][i];
    return sum;
}
```

```
void diagonalTraversal(int n, int a[n][n])
{
    int s, i, j;
    for (s = 0; s <= 2 * n - 2; s++)
    {
        if (s % 2 == 0)
        {
            i = (s < n) ? s : n - 1;
            j = s - i;
            while (i >= 0 && j < n)
            {
                printf("%d ", a[i][j]);
                i--;
                j++;
            }
        }
        else
        {
            j = (s < n) ? s : n - 1;
            i = s - j;
            while (j >= 0 && i < n)
            {
                printf("%d ", a[i][j]);
                i++;
                j--;
            }
        }
    }
    printf("\n");
}
```

```
int main()
{
    int n, i, j, choice;

    printf("Enter the order of square matrix: ");
    scanf("%d", &n);

    int a[n][n];
```

```

printf("Enter the matrix elements:\n");
for (i = 0; i < n; i++)
    for (j = 0; j < n; j++)
        scanf("%d", &a[i][j]);

do
{
    printf("\n1. Check distinct main diagonal elements\n");
    printf("2. Sum of main diagonal elements\n");
    printf("3. Diagonal traversal\n");
    printf("4. Exit\n");
    printf("Enter your choice: ");
    scanf("%d", &choice);

    switch (choice)
    {
        case 1:
            if (distinctDiagonal(n, a))
                printf("All main diagonal elements are distinct.\n");
            else
                printf("Main diagonal elements are not distinct.\n");
            break;

        case 2:
            printf("Sum of main diagonal elements: %d\n",
                sumDiagonal(n, a));
            break;

        case 3:
            printf("Diagonal traversal: ");
            diagonalTraversal(n, a);
            break;

        case 4:
            printf("Exiting program...\n");
            break;

        default:
            printf("Invalid choice. Please enter 1 to 4.\n");
    }
} while (choice != 4);

return 0;
}

```

Compilation

```

(kali@kali)-[~/hdoc35]
$ gcc hdoc35.c -o hdoc35

```

Execution 1

```

(kali@kali)-[~/hdoc35]
$ ./hdoc35
Enter the order of square matrix: 3
Enter the matrix elements:
1 2 3
4 5 6
7 8 9

1. Check distinct main diagonal elements
2. Sum of main diagonal elements
3. Diagonal traversal
4. Exit
Enter your choice: 1
Main diagonal elements: 1 5 9
All the main diagonal elements are distinct.

```

Execution 2

```
1. Check distinct main diagonal elements
2. Sum of main diagonal elements
3. Diagonal traversal
4. Exit
Enter your choice: 2
Sum of main diagonal elements: 15
```

Execution 3

```
1. Check distinct main diagonal elements
2. Sum of main diagonal elements
3. Diagonal traversal
4. Exit
Enter your choice: 3
Diagonal traversal: 1 2 4 7 5 3 6 8 9
```

Execution 4

```
1. Check distinct main diagonal elements
2. Sum of main diagonal elements
3. Diagonal traversal
4. Exit
Enter your choice: 4
Exiting program...

(kali㉿kali)-[~/hdoc35]
$
```